Chapter 1: Introduction to Operating Systems

By: Ishan Shrestha

1.1 Introduction to Operating Systems

An operating system (OS) is a fundamental software that acts as an intermediary between computer hardware and software applications, providing essential services, managing resources, and abstracting hardware complexities to enable efficient and user-friendly interaction with the computer system.

- **Abstraction:** Hardware is complex. The OS hides this complexity from application developers.
- **Resource Management:** decides which application gets to use the CPU when, how much memory each application can use, and how data is read from and written to storage devices. This ensures fair access, prevents conflicts, and optimizes system performance.
- **Providing Services:** Beyond just connecting software and hardware, the OS provides a set of essential services to applications. These include:
  - **File Management:** Creating, reading, writing, and deleting files.
  - **Process Management:** Starting, stopping, and managing the execution of programs.
  - **I/O Management:** Handling input from keyboards, mice, and output to screens and printers.
  - **Networking:** Facilitating communication over networks.

1.2 OS as an Extended Machine and OS as a resource manager.

- **OS as an Extended Machine**

  Interacting with hardware is complex, each hardware has its own way of being controlled. OS as extended machine creates a simpler and abstract view of hardware for application and users and presents an easier way to interact.

  Here's how it does it:

  - **Abstraction:** OS turns complex hardware operations into simpler commands. **Example:** Instead of specifying the cylinder, sector and track of disk to save a document, it just says save file to

OS to let it figure out the communication with the disk.
  - **Convenience:** OS provides a simpler, user-friendly interface that makes it convenient for programmers to develop applications without know the underlying workings of hardware.
  - **Consistency:** Regardless of specific brands and models of hardware, OS provides a simple consistent interface to the application to interact with hardware, and it handles how to handle the exchange.

- **OS as a resource manager**

  A computer system has a finite set of resources:

  - **CPU (Processor):** The brain that executes instructions.
  - **Memory (RAM):** Where programs and data are temporarily stored.
  - **I/O Devices:** Keyboards, mice, displays, printers, network cards, hard drives, SSDs, etc.
  - **Files:** Persistent storage on disk.

  In modern computing, it is necessary for OS to manage those resources among multiple applications and users simultaneously to efficiently and fairly provide the resources to the applications.

  Key responsibilities of the OS as a resource manager:

  - **Allocation:** decides which programs get to use resources and when. For instance, which program gets to use CPU at a particular instance and the memory to run on.
  - **Protection:** Ensuring another program doesn't interfere with one program's resource while it is using it, and it protects the system against malicious software programs.
  - **Scheduling:** decides the order and duration of time a program uses the resources.
  - **Accounting:** Tracks the use of memory by the programs and billing for multiple users.

  Note: THINK OF IT AS TRAFFIC POLICE, WHO MANAGES WHICH LANE GOES AT WHAT TIME.

1.3 History of Operating Systems

- **Early Systems (1940s - mid-1950s): No Operating Systems**
  - **Concept:** Computers were enormous, expensive machines. Programmers interacted directly with the bare hardware.
  - **How it worked:** Programs were written in machine language or assembly, often loaded manually using switches or paper tape. Each job involved setting up the machine, running the program, and then tearing down the setup.
  - **Problems:** Extremely inefficient. A lot of CPU time was wasted during setup and teardown between jobs. Only one user could use the machine at a time. No abstraction or resource management.

- **Batch Systems (mid-1950s - mid-1960s)**
  - **Concept:** To improve efficiency, similar jobs were batched together and run sequentially without manual intervention between jobs.
  - **How it worked:** A "monitor" program (a rudimentary OS) would reside in memory. Programmers would submit jobs on punch cards or magnetic tape. The monitor would load and execute jobs one after another.
  - **Key Innovation:** Introduction of simple resident monitors to automate job transitions. This was the very beginning of OS functionality.
  - **Problems:** Still inefficient for individual users, as they had to wait for their turn in the batch queue. No interaction with the running program. CPU could still be idle during I/O operations.

- **Multiprogramming Systems (mid-1960s - early 1970s)**
  - **Concept:** The major breakthrough! The idea was to keep the CPU busy by allowing multiple programs to reside in memory at the same time. When one program needed to wait for an I/O operation (which is slow), the OS could switch the CPU to another ready program.
  - **How it worked:** Requires memory protection (so programs don't interfere with each other) and CPU scheduling mechanisms.
  - **Key Innovations:** CPU scheduling, memory management, I/O handling becoming more sophisticated. This significantly improved CPU utilization.
  - **Problems:** Still primarily batch-oriented or for large, shared mainframes. Interactive computing was limited.

- **Time-Sharing Systems (late 1960s - present)**
  - **Concept:** An extension of multiprogramming, designed to provide interactive computing. Each user gets a small slice of CPU time in a rapid, round-robin fashion, giving the illusion that they have dedicated access to the machine.
  - **How it worked:** Requires sophisticated CPU scheduling (like Round Robin), virtual memory, and file systems to manage user data. Users interact via terminals.
  - **Key Innovations:** Direct user interaction, virtual memory, rise of command-line interfaces. Unix emerged from this era.
  - **Examples:** CTSS, Multics, Unix.

- **Personal Computer Systems (1970s - present)**
  - **Concept:** Affordable computers for individual users. Early PCs were single-user, single-tasking.
  - **How it worked:** Initially simple OS like CP/M, then MS-DOS. Focus shifted to ease of use for a single user.
  - **Key Innovations:** Rise of graphical user interfaces (GUIs) with systems like Apple Macintosh and later Windows. From single-tasking, they evolved into multitasking, multi-user (though typically one active user at a time) systems.

- **Distributed Systems (1980s - present)**
  - **Concept:** Multiple independent computers working together as a single, integrated system.
  - **How it worked:** Requires network communication, distributed file systems, and mechanisms for distributed process management.
  - **Key Innovations:** Network operating systems, distributed computing.

- **Mobile and Embedded Systems (1990s - present)**
  - **Concept:** Operating systems designed for devices with limited resources, specific functionalities, and often real-time constraints.
  - **How it worked:** Optimized for low power, small memory footprint, touch interfaces, and specific hardware.
  - **Examples:** VxWorks, FreeRTOS, Android, iOS.

1.4 Types of Operating System

- **Distributed Systems**
  - **Concept:** Instead of a single powerful computer handling all tasks, a distributed system consists of multiple independent computers (nodes) interconnected by a network, which work together to achieve a common goal or provide a unified service. To the user, it often appears as a single, coherent system.
  - **Why they emerged:**
    - **Scalability:** Easier to add more computing power by adding more nodes.
    - **Reliability/Fault Tolerance:** If one node fails, others can take over.
    - **Resource Sharing:** Sharing resources (like printers, databases) across a network.
    - **Geographic Distribution:** Users or data can be spread across different locations.
  - **How they work (OS challenges):**
    - **Network Transparency:** Hiding the fact that resources are located on different machines.
    - **Concurrency Control:** Managing simultaneous access to shared data across multiple nodes.
    - **Fault Tolerance:** Designing systems that can recover from individual node failures.
    - **Security:** Protecting data and resources across a network.
    - **Examples of OS or OS components:** Network Operating Systems (NOS) like Novell NetWare (historically), Windows Server, Linux (acting as servers in a distributed environment), distributed file systems (like NFS, HDFS), cloud computing platforms (which are essentially large distributed systems).
  - **Analogy:** Think of a large company where different departments (nodes) handle specific tasks, but they all work together using shared resources and communicate constantly to deliver a final product.

- **Mobile and Handheld Systems**
  - **Concept:** Operating systems designed specifically for portable devices like smartphones and tablets.
  - **Key Characteristics:**
    - Resource Constraints: Optimized for low power consumption, smaller memory footprints, and less powerful CPUs compared to desktop PCs.
    - **Touch-based User Interfaces:** Heavily rely on touch input rather than keyboard/mouse.
    - **Connectivity:** Integrated Wi-Fi, cellular data, Bluetooth, GPS.
    - **App Ecosystem:** Support for a large number of specialized applications.
    - **Security & Privacy:** Robust mechanisms for data protection and user privacy are critical.
  - **Examples:** Android, iOS, Windows Phone (historically).
  - **Evolution from PC OS:** While they share core OS concepts, they are highly optimized and tailored for the unique interaction models and hardware limitations of mobile devices.

- **IoT and Embedded Systems**
  - **Concept:** Operating systems designed for specialized, often resource-constrained devices that perform a dedicated function within a larger system. "IoT" (Internet of Things) devices are a subset of embedded systems that are connected to the internet.
  - **Key Characteristics:**
    - **Small Footprint:** Very small code size, minimal memory requirements.
    - **Specific Task:** Designed to do one job extremely well (e.g., control a washing machine, manage a smart thermostat, collect sensor data).
    - **Real-time Capabilities (often):** May need to respond to events within strict deadlines (we'll cover Real-Time OS separately next).
    - **Low Power:** Crucial for battery-operated devices.
    - **No User Interface (often):** Many embedded systems operate without a direct user interface, or with a very minimal one.
    - **Robustness:** Must be extremely reliable as they often operate autonomously for long periods.
  - **Examples:** FreeRTOS, VxWorks, QNX, Embedded Linux (tailored versions for specific hardware). Think of the OS in your smart TV, microwave, car's engine control unit, smart home devices, industrial robots.

- **Real-Time Systems (RTS / RTOS)**
  - **Concept:** Operating systems where the correctness of an operation depends not only on the logical result but also on the *time* at which the result is produced. They have strict deadlines for tasks.
  - **Types:**
    - **Hard Real-Time Systems:** Missing a deadline is a catastrophic failure (e.g., flight control systems, medical life-support systems, anti-lock brakes).

        - **Soft Real-Time Systems:** Missing a deadline is undesirable but not catastrophic; it leads to degraded performance (e.g., multimedia streaming, virtual reality, online gaming).
    - **Key Characteristics:**
        - **Predictability:** The most important feature. They must guarantee that tasks will complete within specified timeframes.
        - **Determinism:** Given the same inputs, the system will always produce the same output in the same amount of time.
        - **Fast Context Switching:** To quickly switch between tasks to meet deadlines.
        - **Priority-Based Scheduling:** Tasks with higher priority always get preference.
        - **Minimal Latency:** Low response time to events.
    - **Examples:** VxWorks, QNX, RTLinux, FreeRTOS.
    - **Application:** Industrial control, robotics, aerospace, medical devices, automotive systems.

- **Smart-Card Systems**
    - **Concept:** Extremely small, highly specialized operating systems embedded within smart cards (e.g., credit cards, SIM cards, ID cards).
    - **Key Characteristics:**
        - **Tiny Footprint:** Minimal memory (often KBs) and processing power.
        - **Security:** Paramount. Designed with very strong security features to protect sensitive data (e.g., cryptographic keys).
        - **Limited I/O:** Typically communicate through a serial interface with a card reader.
        - **Single-Purpose:** Designed for specific tasks like secure authentication, payment processing.
    - **Examples: J**ava Card OS, MULTOS, or custom-built micro-kernels.

1.5 Operating System Components

- **Kernel:**

  Kernel is the most fundamental part of the OS, It is the first program loaded into the memory when the computer starts and remains in the memory as long as the computer is running.

    - **Analogy:** If the entire computer system were a car, the kernel would be its engine. It handles all the fundamental operations that make the car move and function, even if you, as the driver, are interacting with the steering wheel (shell) or listening to music (applications).
    - **Key Responsibilities of the Kernel:**
        - **Process Management:**
            - **Creation and Termination:** It is responsible to create processes and delete them after completion.
            - **Scheduling:** It is responsible to manage which process gets the CPU and for how long.

        - **Synchronization and Communication:** Provide mechanism to coordinate and share data with each other.
    - **Memory Management:**
        - **Allocation and Deallocation**
        - **Protection:** Ensuring one process doesn't access and corrupt memory space of other or kernel itself.
        - **Virtual Memory Management**
    - **Device Management (I/O Management):**
        - **Controlling Hardwares:** The OS is responsible to interact directly with the hardwares.
        - **Device Drivers:** The OS usually contains or loads device drivers, which are small programs used to OS commands into hardware operations.
        - **Buffering and Spooling:** Manage Data flow between slow devices and fast CPU.
    - **Interrupt Handling:**
        - **Responds to live events**

- **Shell:**
    - **Definition:** The shell is a command line interface or graphical line interface which provides an interface to interact directly with the OS.
    - **Types of Shell:**
        - **Command-line Interface shells:** In this type of shell, we manually write the commands in the shell and the OS executes them.
            - **Examples:** Bash, CMD, Powershell, etc
            - **Advantages:** Powerful, efficient for repetitive tasks, uses fewer resources, etc
            - **Disadvantages:** Requires to memorize commands
        - **Graphical User Interface shells:** The shell is used through interacting with windows, icons,menu and pointers. The GUI translates the graphical actions into system calls.
            - **Examples:** Windows Explorer, GNOME Shell, etc
            - **Advantages:** Easy and Intuitive to learn
            - **Disadvantages:** Requires more resources
    - **Key Functions of Shell:**
        - Command Interpretation
        - Program Execution
        - Input/Output Redirection
        - Environment Management

- **Utilitites:**
    - The utilities are the user level program, makes system calls to perform a specific tasks.
    - **Key characterstics of Utilities:**
        - **System Management:**
            - **File Management:** Tools for file management (copy, moving deleting, renaming)
            - **Disk Management:** Tools for formatting disks, checking disk errors, usagae and check partiton of the disk.
            - **Process Management:** Tools for veiwing and managing running processes.

        - **Network Utilities**
    - **Performance Monitoring**
    - **Security and Permission**
    - **Archiving and Compression**
    - **Basic Text Editors**

- **Applications:**
    - These are the programs that perform highly specific tasks that are not directly related to the computer functioning.
    - **Key Characterstics:**
        - **User-Focused Tasks**
        - **Run in user mode**
        - **Dependance on OS**
        - **Variety and Ecosystem**

- **Analogy:**
    - **Kernel:** The engine (makes the car run).
    - **Shell:** The dashboard and steering wheel (how you control the car).
    - **Utilities:** The toolkit (for maintaining and fixing the car).
    - **Applications:** These would be things *inside* the car that serve your specific purpose for driving – like the navigation system (Google Maps), the radio (Spotify), or a snack box (a productivity app for your journey). They help you achieve your *goal* using the car.

1.6 Types of OS kernel:

- **Monolithic Kernel:**
    - All operating system services and components are packed into a single, large executable program.
    - **Key Characterstics:**
        - **Single Address Space:** Faster communication as all kernel services are in the same memory.
        - **Large Size**
        - **Excellant Performance**
    - **Advantages:**
        - High Performance
        - Simpler Design
    - **Disadvantages:**
        - Lack of Modularity

        - Stability/Reliablity Issue
        - Difficult to debug

- **Microkernel**
    - Kernel is kept as minimal as possible, only contains absolute essential functions needed to run the operating system. All other traditional functions of OS like file systems are moved out of kernel and run as seperate, isolated user-level processes.
    - **Key Features:**
        - **Minimalistic core:**
            - Interprocess communication
            - Low level process management
            - Memory Management
            - Hardware Abstraction
        - **Modular and Extensible:**
        - **Increased Reliablity/Stability**
    - **Advantages:**
        - Improved Stability/Reliability
        - Enhanced Security
        - Modularity
    - **Disadvantages:**
        - Performance Overhead
        - Increased Complexity (for communication)

- **Nano Kernel:**
    - Even smaller kernel, reducing functionality to bare minimun necessary to provide fundamental services.
    - **Key Features:**
        - **Extreme Minimalistic:**
            - Interrupt Handling
            - Highly Specialized
            - Performance vs Complexity

- **Layered Kernel:**
    - Layered Kernel architecture is an approach to structuring the OS as s heirarchy of layers, each layer builds upon the services of layers below it, and provides services for layers above it.
    - **Key Features:**
        - **Heirarchical Structure:**
            - Level 0 (innermost): It deals with hardware, lowest level of abstraction.
            - Higher Layers: Builds upon services of lower layers.
            - Outer Layers: User facing services.
        - **Clear Abstraction:**
        - **Modularity**

        - **Debugging**
    - **Advantages:**
        - Easy debug and testing
        - Modularity
        - Better Maintainability
    - **Disadvantages:**
        - Performance Overhead
        - Difficulty defining layer

- **Hybrid Kernel:**
    - Is an approach in which best aspects of monolithic and microkernels are combined.
    - **Key Features:**
        - Core services in Kernel space
        - Modular and Loadable components
        - Performance Optimization
    - **Advantages:**
        - Good Balance
        - Flexibility
        - Robustness
    - **Disadvantages:**
        - Increased Complexity
        - Still prone to kernel crash

- **Exokernel:**
    - provide minimal abstractions and instead exposing hardware resources directly to user-level applications in a secure and protected manner.
    - **Key Features:**
        - Exposes Hardware
        - Application-specific Abstraction
        - Performance and Flexibitly
    - **Advantages:**
        - Maximum Flexibility
        - Potentially Higher performance
    - **Disadvantages:**
        - Immense Deployment Complexity
        - Less Portability

1.7 System calls, shell commands, shell programming

- **System Calls:**

  System calls are the interface between user level application and the OS kernel.

  - **Why is it necessary?**

    As, application run in less privilaged 'user mode', it cannot access kernel features, so it requests kernel to provide those services through system calls.

  - **How System Calls Work:**
    i. **Request from User Program:** A user-level program needs to perform an operation that requires kernel privileges or resources (e.g., read a file, create a new process, allocate memory, send data over a network, access a device).
    ii. **Library Function Call:** Typically, applications don't call system calls directly. Instead, they call standard **library functions** (e.g., printf() for printing, fopen() for opening a file, malloc() for memory allocation in C/C++).
    iii. **Wrapper/Stub:** These library functions act as "wrapper functions" or "stubs." When called, they prepare the necessary arguments for the system call and then execute a special machine instruction (often an interrupt or a trap instruction).
    iv. **Mode Switch (User Mode to Kernel Mode):** This special instruction causes a hardware interrupt, which traps the CPU into kernel mode. Control is immediately transferred to a specific entry point within the kernel.
    v. **Kernel Execution:** The kernel identifies which system call was requested (based on a system call number passed as an argument) and executes the corresponding kernel-level service routine.
    vi. **Return to User Program:** Once the kernel service completes, it returns control and any results (e.g., data read from a file, a success/failure code) back to the user program. The CPU switches back from kernel mode to user mode.

- **Shell Commands:**

  A shell command is a command that a user types to the shell to tell Operating system to perform a specific task.

  - **How Shell Commands Work:**
    i. **User Input:** You type a command (e.g., ls -l in Linux, or dir in Windows) at the shell prompt and press Enter.
    ii. **Shell Interpretation:** The shell (e.g., Bash, Cmd, PowerShell) interprets what you've typed. It identifies:
      - **The command:** Is it an internal shell command (built-in)? Is it an external program located on the system's PATH?
      - **Arguments/Options:** Any additional information or flags you provided (e.g., -l in ls -l which means "long listing format").

    iii. **Command Execution:**
      - **Built-in Commands:** If it's a built-in shell command (e.g., cd for "change directory"), the shell itself executes the command directly without invoking a separate program. These

are often used for efficiency or because they modify the shell's own environment.
    - **External Programs:** If it's an external program (which most commands are, like ls, grep, mkdir), the shell does the following:
        - It searches for the executable file of that program in predefined directories (listed in the PATH environment variable).
        - Once found, the shell makes **system calls** to the kernel to:
            - **Create a new process** to run that program (fork() in Unix-like systems).
            - **Load the program's code** into the new process's memory space (exec() in Unix-like systems).
            - **Start the execution** of the program.
iv. **Output:** The program runs, and any output it generates is typically displayed back in the shell window.
v. **Shell Continues:** Once the program finishes, control returns to the shell, and you see the prompt again, ready for your next command.

- **Shell Scripting:**

Act of writing sequences of shell commands into a file such that it can be executed as a single program.

- **Basic Concepts of Shell Programming:**

Shell scripts typically use the syntax of a particular shell, most commonly **Bash** (Bourne Again SHell) on Linux and macOS, but also PowerShell on Windows.

i. **Shebang Line (#!):** The very first line of a shell script usually specifies which interpreter should be used to execute the script.
    - Example: #!/bin/bash (for a Bash script)
ii. **Comments:** Lines starting with # are comments and are ignored by the interpreter.
iii. **Variables:** You can store data in variables.
    - Example: name="Alice", count=10
iv. **Input/Output:**
    - **Echo:** echo "Hello, $name!" (prints text to the screen)
    - **Read:** read -p "Enter your age: " age (reads user input)
v. **Control Flow (Crucial for Logic):**
    - **Conditionals (if, elif, else):** Execute different blocks of code based on conditions.
    - **Loops (for, while):** Repeat commands multiple times.
vi. **Command Execution:** Any standard shell command (like ls, mkdir, cp) can be used directly within a script. The output of one command can often be piped as input to another.
    - Example: ls -l | grep "txt" (lists files, then filters for lines containing "txt")
vii. **Arguments:** Scripts can accept arguments when they are executed ($1, $2, etc., refer to the first, second argument).
    - Example: If script myscript.sh contains echo "First arg: $1", then running myscript.sh hello would print "First arg: hello".

1.8 POSIX Standard

POSIX Standard stands for **P**ortable **O**perating **S**ystem **I**nterface for Uni**x,** It's a set of standards set by IEEE (Institute of Electrical and Electronics Engineers) that deifnes a standard interface for operating systems, primarily focusing on Unix-like operating systems.

Why the need?

When developer wrote a program for a system, they needed different codes for another system. So, there needed a standard that operating systems can follow.

What POSIX define?

POSIX specifies standard interfaces for:

a. **System Calls:** It defines the syntax and semantics of a wide range of system calls that an application can expect to find on a POSIX-compliant OS. This includes calls for:
   - Process management (fork(), exec(), wait(), exit())
   - File I/O (open(), read(), write(), close())
   - Directory operations (mkdir(), rmdir())
   - Pipes and IPC (pipe())
   - Signals (kill())
   - Time and date functions (time())

b. **Standard Library Functions:** It defines common C library functions that are closely related to OS services.

c. **Shell and Utilities:** It specifies the behavior and syntax of common command-line utilities (like ls, cat, grep, awk, sed) and the shell itself (Bash is largely POSIX-compliant). This ensures that shell scripts written on one POSIX system will behave consistently on another.

d. **Header Files:** It defines standard header files (e.g., <unistd.h>, <fcntl.h>) that programs should include to access these functionalities.

Key Goals and Benefits of POSIX:

- **Portability**: The primary goal. Software written to the POSIX standard can be easily compiled and run on any POSIX-compliant operating system with minimal or no modifications. This saves development time and cost.
- **Interoperability**: Different systems can more easily work together.
- **Reduced Vendor Lock-in**: Organizations are not tied to a single vendor's specific Unix implementation.
- **Consistency**: Provides a predictable environment for developers.

1.9 Bootloader, MBR/GPT, UEFI and legacy boot

- **The Bootloader**

Definition: A bootloader is a small program that resides in a specific location on a specific device. It's primary task is to load the operating system kernel into the memory and hand control over to it.

The Boot Process (Simplified High-Level Overview):

a. **Power On:** You press the power button.
b. **Firmware Initialization (BIOS/UEFI):** The computer's built-in firmware (either BIOS or UEFI) starts up. It performs a **P**ower-**O**n **S**elf-**T**est (POST) to check if basic hardware components (CPU, memory, graphics card) are working.
c. **Boot Device Selection:** The firmware then looks for a bootable device (e.g., hard drive, SSD, USB, CD/DVD) in a predefined order (which you can usually configure in the firmware settings).
d. **Bootloader Execution:** Once a bootable device is found, the firmware reads a tiny, specific part of that device (e.g., the Master Boot Record or the EFI System Partition) and loads the **bootloader** into a small portion of RAM.
e. **Bootloader's Job:** The bootloader then takes over. Its main tasks typically include:
    - **Locating the OS Kernel:** It knows where the operating system kernel file (and sometimes related initial files like an initial RAM disk) is located on the storage device.
    - **Loading the Kernel:** It loads the kernel (which can be quite large) into the main memory (RAM).
    - **Passing Control:** Finally, it passes control (the CPU's execution) to the loaded OS kernel.
f. **OS Takes Over:** The OS kernel then initializes itself, loads device drivers, starts system services, and eventually presents you with the login screen or desktop environment.

Multi-Stage Bootloaders:

Modern operating systems often use **multi-stage bootloaders**. This means the bootloader isn't a single program but a sequence of small programs:

- **Stage 1 (Primary Bootloader):** Very small, typically located in the MBR (Master Boot Record) or a specific part of the EFI System Partition. Its only job is to load the next stage. It's often limited in size due to where it resides.
- **Stage 2 (Secondary Bootloader):** Larger and more sophisticated. It can understand file systems, display boot menus (allowing you to choose which OS to boot if you have multiple), load kernel modules, and finally load the OS kernel.

- **MBR/GPT**
    - **MBR (Master Boot Record):**

      The Master Boot Record is a special 512-byte sector located at the very beginning of a hard drive (the first sector).

        - It contains two main things:
            1. **Bootloader Code:** A small piece of executable code (the "Stage 1" bootloader, as we discussed earlier) that tells the computer how to proceed with the boot process.
            2. **Partition Table:** A table that stores information about the partitions on the disk.
        - **Limitations:**
            - **max disk size:** 2TBs
            - **Max partitions:** 4 primary partitions
            - **No redundancy:** Stored in single location, if it becomes corrupted, entire disks becomes unbootable.
            - **Older Boot model**

- **GPT (Guid Partition Table):**

  GPT is a newer and more robust partitioning scheme, developed as part of the UEFI (Unified Extensible Firmware Interface) standard.

  Advantages:

  - **Larger Disk Size Support:** GPT can handle extremely large disk sizes, virtually unlimited (up to 9.4 Zettabytes, far beyond current practical limits).
  - **More Partitions:** It supports up to **128 primary partitions** by default (and can be extended), eliminating the need for extended and logical partitions.
  - **Redundancy and Reliability:** GPT stores multiple copies of the partition table across the disk, including a backup copy at the end of the disk. This makes it much more resilient to corruption. It also includes CRC (Cyclic Redundancy Check) values to detect errors in the header and partition table.
  - **Globally Unique Identifiers (GUIDs):** Each partition has a globally unique identifier, which helps prevent name collisions and simplifies management.
  - **Modern Boot Mode:** It is natively associated with **UEFI boot mode**.

- **UEFI (Unified Extensible Firmware Interface) and Legacy Boot (BIOS)**

  - **Legacy Boot (BIOS)**

    BIOS is the older, traditional firmware standard, dating back to the early days of personal computing (1980s).

    How it works (Legacy Boot Process):

    i. **POST:** When you power on, the BIOS performs the Power-On Self-Test (POST).
    ii. **Boot Device Order:** It checks the configured boot device order.
    iii. **MBR Check:** For a hard drive, it looks for the **Master Boot Record (MBR)** at the very first sector of the designated boot disk.
    iv. **Loads MBR Bootloader:** It loads the 512-byte MBR bootloader code (Stage 1) into memory at a specific fixed address and executes it.
    v. **Bootloader Continues:** This MBR bootloader then takes over to find and load the rest of the operating system (e.g., Stage 2 bootloader, then the OS kernel).

    Limitations of BIOS:

    - **16-bit Mode:** BIOS operates in 16-bit processor mode, which is very limiting in terms of memory access and processing capabilities.
    - **Limited Disk Support:** Tied to MBR's 2TB disk size limit and 4 primary partitions.
    - **Slower Boot Times:** The process is generally slower compared to UEFI.
    - **No GUI:** Typically text-based interface for settings.

- **UEFI (Unified Extensible Firmware Interface)**

  UEFI is the modern successor to BIOS, developed in the mid-2000s and now dominant on most new computers. It was designed to overcome the limitations of BIOS.

  How it works (UEFI Boot Process):

  i. **POST:** UEFI performs similar power-on self-tests.
  ii. **UEFI Boot Manager:** Instead of scanning for an MBR, UEFI has a sophisticated **Boot Manager**. This manager reads **boot entries** stored in NVRAM (Non-Volatile Random-Access Memory) on the motherboard. Each entry points to a specific bootloader file (an .efi executable) located on an **EFI System Partition (ESP)** on a **GPT-formatted disk**.
  iii. **Loads EFI Bootloader:** UEFI loads the .efi bootloader file directly from the ESP into memory and executes it.
  iv. **Bootloader Continues:** This EFI bootloader (e.g., GRUB EFI, Windows Boot Manager) then loads the OS kernel.

  Advantages of UEFI:

  - **32-bit or 64-bit Mode:** UEFI can operate in more capable processor modes, allowing it to access more memory and run more complex code.
  - **Larger Disk Support:** Natively supports **GPT** partitioning, enabling support for disks larger than 2TB and more partitions.
  - **Faster Boot Times:** Generally, UEFI boots significantly faster.
  - **Graphical Interface:** Often provides a graphical user interface (GUI) for settings, sometimes with mouse support.